

深圳大学《并行计算》期末冲刺综合模拟卷

PPT重点版 - 闭卷 - 120分钟 - 共10题 - 100分 - 试题卷

说明：第2、3、4、5、6章各2题；每题10分；请按闭卷要求作答，写出关键步骤、理由和必要公式。

建议时间分配：每题约12分钟；程序题先找数据竞争、变量作用域、同步和边界条件；算法题先写步骤再写关键结果。

第1题 [第2章 并行硬件与并行软件 | 概念解释 | 10分]

说明 cache 行、空间局部性与二维数组遍历性能之间的关系。为什么在 C/C++ 的行优先存储中，按行访问二维数组通常比按列访问更快？

比较 UMA 和 NUMA 的特点，并说明 MPI、Pthreads、OpenMP 分别更偏向哪类内存编程模型。

第2题 [第2章 并行硬件与并行软件 | 计算与分析 | 10分]

某程序串行时间 $T_1=N$ ，并行时间 $T_p=N/p+5$ 。

- (1) 求固定负载 N 时的加速比 S 和效率 E ，并讨论 p 增大时的上限。
- (2) 若固定并行运行时间为 T ，反解可处理的问题规模 N ，并求固定时间加速比。
- (3) 在 8×8 二维环绕网络中，从 $(1,1)$ 到 $(6,7)$ 的最短距离是多少？给出一条最短路径。

第3题 [第3章 MPI 分布式内存编程 | 程序改错 | 10分]

下面 MPI 片段用于进程 0 向进程 1 发送字符串。指出至少 4 处问题并给出修改理由；同时说明为什么 MPI 程序计时更适合用 MPI_Wtime 而不是 clock。

```
MPI_Init(NULL, NULL);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);

if (rank == 0) {
    char msg[] = "hello";
    MPI_Send(msg, strlen(msg), MPI_CHAR, 1, 0, MPI_COMM_WORLD);
} else if (rank == 1) {
    char buf[100];
    MPI_Recv(buf, 100, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);
    printf("%s\n", buf);
}

elapsed = clock();
MPI_Finalize();
```

第4题 [第3章 MPI 分布式内存编程 | 算法执行过程 | 10分]

用 MPI 棋盘划分实现 Cannon 矩阵乘法。设处理器构成 $q \times q$ 网格，矩阵 A 、 B 、 C 均按块划分。

- (1) 说明棋盘划分和初始对准过程。
- (2) 说明每一轮中 $P(i,j)$ 的本地计算、 A 块移动和 B 块移动。
- (3) 写出 q 轮后 $C(i,j)$ 所累加的块乘积形式。

第5题 [第4章 Pthreads 共享内存编程 | 程序分析与改错 | 10分]

下面代码尝试用互斥锁和条件变量实现 barrier。回答：它表达了什么同步思想？作为可复用 barrier 是否可靠？说明原因，并给出改进思路。答案中必须解释“虚假唤醒”以及为什么 pthread_cond_wait 应放在 while 循环中。

```
pthread_mutex_lock(&mutex);
counter++;
if (counter == thread_count) {
    counter = 0;
    pthread_cond_broadcast(&cond_var);
} else {
    pthread_cond_wait(&cond_var, &mutex);
}
pthread_mutex_unlock(&mutex);
```

第6题 [第4章 Pthreads 共享内存编程 | 程序优化与概念解释 | 10分]

多线程矩阵-向量乘法中，不同线程分别写 y 数组中相邻的不同元素。

- (1) 解释什么是伪共享。它为什么会降低性能？
- (2) 给出两种减少伪共享的优化方法。
- (3) 解释读写锁的适用场景，并说明什么是写者饥饿。

第7题 [第5章 OpenMP 共享内存编程 | 程序改错 | 10分]

下面 OpenMP 程序片段用 Leibniz 级数近似 pi。指出数据竞争或变量作用域问题，写出正确的 OpenMP 写法，并说明 reduction、private、critical、atomic 在本题中的适用性。

```
double sum = 0.0;
double factor = 1.0;

#pragma omp parallel for num_threads(10)
for (long i = 0; i < n; i++) {
    factor = (i % 2 == 0) ? 1.0 : -1.0;
    sum += factor / (2 * i + 1);
}
pi = 4.0 * sum;
```

第8题 [第5章 OpenMP 共享内存编程 | 算法执行过程 | 10分]

回答 OpenMP 中两个重点算法问题。

- (1) 写出 PSRS 正则采样排序的完整步骤。以 $\alpha=(16,10,6,4,14,8,5,13,11,18,17,12,9,2,3,7,1,15)$, $p=3$ 为例，给出局部排序、正则采样、样本排序、主元选择和最终分桶归并的关键结果。本题按从 0 开始取第 p 、 $2p$ 个样本作为主元。
- (2) 对递归归并排序使用 OpenMP task 时，为什么外层通常要 parallel + single？为什么归并前要 taskwait？

第9题 [第6章 CUDA GPU 编程 | 程序改错与索引公式 | 10分]

下面 CUDA kernel 试图让每个线程计算 C 矩阵的一个元素。指出索引、越界和执行模型相关问题，并给出正确的 row/col 公式和边界判断。答案中解释 grid、block、thread、warp、SM 的含义。

```
__global__ void MatMul(double* A, double* B, double* C, int n) {
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    int col = blockIdx.y * blockDim.y + threadIdx.y;
    double sum = 0.0;
    for (int k = 0; k < n; k++) {
        sum += A[row * n + k] * B[k * n + col];
    }
    C[row * n + col] = sum;
}
```

第10题 [第6章 CUDA GPU 编程 | 程序优化与算法过程 | 10分]

回答 CUDA 同步与优化问题。

- (1) 说明 shared memory 和 warp shuffle 在块内归约中的作用。若一个 block 含多个 warp，为什么常需要两级归约？
- (2) 为什么 __syncthreads() 只能同步同一个 block 内的线程？如果需要跨 block 的全局同步，通常如何处理？
- (3) 简述 bitonic sort 的核心过程：如何构造双调序列，并通过比较交换归并为有序序列。